

Clase Básica: Crea una clase Producto en PHP con propiedades public \$nombre y public \$precio. Crea un método aplicarDescuento(\$porcentaje).

```
<?php

class Producto
{
    public $nombre;
    public $precio;

    public function aplicarDescuento($porcentaje)
    {
        $this->precio = $this->precio - ($this->precio *
$porcentaje / 100);
    }
}

?>
```

Constructores: Convierte la clase anterior para que use __construct y asegúrate de que el precio no pueda ser negativo mediante una lógica simple.

```
<?php

class Producto
{
    public $nombre;
    public $precio;

    public function __construct($nombre, $precio)
    {
        $this->nombre = $nombre;

        if ($precio < 0) {
            $this->precio = 0;
        } else {
            $this->precio = $precio;
        }
    }

    public function aplicarDescuento($porcentaje)
    {
        $this->precio = $this->precio - ($this->precio *
$porcentaje / 100);
    }
}
```

```
?>
```

Ejemplo de Uso

```
$producto = new Producto("Laptop", 1200);
```

```
$producto->aplicarDescuento(10);
```

```
echo $producto->precio;
```

Integración Laravel: Imagina que estás en un Controlador de Laravel. ¿Cómo instanciarías una clase Servicio y llamarías a un método para procesar un pago?

En Laravel es común crear clases de servicio para separar la lógica de negocio de los controladores. Supongamos que existe una clase llamada ServicioPago con un método procesarPago().

Clase de servicio:

```
<?php
```

```
class ServicioPago
{
    public function procesarPago($monto)
    {
        return "Pago procesado por $" . $monto;
    }
}
```

Uso desde un controlador de Laravel:

```
<?php
```

```
namespace App\Http\Controllers;
```

```
use App\Services\ServicioPago;
```

```
class PagoController extends Controller
```

```
{
```

```
public function pagar()  
{  
    $servicio = new ServicioPago();  
  
    $resultado = $servicio->procesarPago(100);  
  
    return $resultado;  
}  
}
```

En este ejemplo, el controlador crea una instancia de ServicioPago, llama al método procesarPago() y devuelve el resultado al usuario. Esto permite mantener el controlador más limpio y facilita la reutilización de la lógica de negocio en otras partes de la aplicación.